

Multiplierless FIR Filter Design with Parallel-Cut Ellipsoid Method

Shifts, adders, and a separation oracle

Wai-Shing Luk

Fudan University

September 25, 2026

Section 1

Agenda



Agenda



Part 1 — The Design Problem

FIR filters · the cost of a multiplier ·
CSD · shift-add sharing



Part 2 — The Engine separation
oracle · ellipsoid update · parallel cuts



Part 3 — Convexity &

Discreteness spectral factorization ·
quantization-aware oracle



Part 4 — Practice three
implementations · results · lessons

Section 2

The Design Problem



FIR Filters and the Cost of a Multiplier

- A FIR filter is a weighted sum:

$$y[t] = \sum_{k=0}^{n-1} h[k] u[t - k].$$

- Its response must satisfy a magnitude mask $L(\omega) \leq |H(\omega)| \leq U(\omega)$.
- In ASIC/FPGA the **multiplier dominates area and power** 🚦
- A **multiplierless** tap replaces each product by **shifts and adds** + -



Canonical Signed Digit (CSD)

- Digits are only $\{-1, 0, +1\}$, and no two adjacent digits are non-zero.
- **Unique**, and with the **fewest non-zero digits** of any signed-digit form:

$$c = \sum_j s_j 2^{e_j}, \quad s_j \in \{-1, 0, +1\}.$$

- Example: $28.5 = +00-00.+0_{\text{csd}} = 32 - 4 + 0.5$ ✨
- A coefficient with d non-zero digits costs $d - 1$ adders; a power of two costs **none**.

+ − Shift-Add Synthesis and Sharing

- A pattern occurring at positions $p < q$ can be shared:

$$\text{pat}_q(x) = \text{pat}_p(x) \gg (q - p).$$

- Pick the pattern maximizing $\text{score} = (\text{nnz} - 1)(\text{occurrences} - 1)$.
- Typical adder savings: **40–60%** for a 64-tap design (32-tap: ~82 cells vs ~110) 
- **MCM** / **CSE** also reduce *adder depth*, hence the critical path.



Three Obstacles

- The magnitude constraint is **not convex** in the impulse response.
 - It must hold at **every** frequency — a **semi-infinite** constraint ∞
 - The CSD quantizer is **discrete**; optimize-then-round can destroy feasibility $\times$
- The ellipsoid method answers all three at once. 🎯*

Section 3

The Engine



The Method Needs Only an Oracle

The oracle Ω , queried at x_0 , either certifies “ $x_0 \in \mathcal{K}$ ” or returns a cut:

$$g^\top(x - x_0) + \beta \leq 0, \quad \beta \geq 0, \quad g \neq 0, \quad \forall x \in \mathcal{K}.$$

- $\beta = 0 \rightarrow$ **central** ○ · $\beta > 0 \rightarrow$ **deep** ▼ · $\beta < 0 \rightarrow$ **shadow** ●
- For $f_j(x) \leq 0$ the cut is free: $(g, \beta) = (\partial f_j(x_0), f_j(x_0))$ ✨
- No need to evaluate all constraints ✓



The Ellipsoid Update

$$\mathcal{E} = \{ x \mid (x - x_c)^\top Q^{-1} (x - x_c) \leq \kappa \}$$

With $\tilde{g} = Qg$, $\omega = g^\top \tilde{g}$, and $\tau = \sqrt{\kappa\omega}$:

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa.$$

- **central:** $\rho = \frac{\tau}{n+1}$, $\sigma = \frac{2}{n+1}$, $\delta = \frac{n^2}{n^2-1}$
- **deep:** $\rho = \frac{\tau + n\beta}{n+1}$, $\sigma = \frac{2(\tau + n\beta)}{(n+1)(\tau + \beta)}$



Parallel Cuts

- A two-sided constraint $l \leq a^\top x + b \leq u$ yields a **pair** of parallel planes sharing a normal g .
- Removes a **slab**, not a half-space $\rightarrow$ faster convergence 🚀
- With $\zeta_0 = \tau^2 - \beta_0^2$, $\zeta_1 = \tau^2 - \beta_1^2$, and $\xi = \sqrt{\zeta_0 \zeta_1 + (\frac{n}{2}(\beta_1^2 - \beta_0^2))^2}$:

$$\sigma = \frac{2\eta}{\tau^2 + \beta_0\beta_1 + \frac{n}{2}(\beta_0 + \beta_1)^2 + \xi}, \quad \rho = \sigma \cdot \frac{\beta_0 + \beta_1}{2}.$$

- Finite even when $\beta_0 + \beta_1 = 0$ 🌟

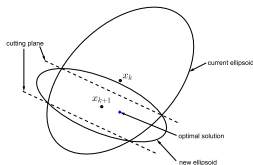


Figure 1: Parallel cuts



Volume Reduction

$$\det Q^+ = (1 - \sigma) \det Q, \quad \frac{\text{vol}(\mathcal{E}^+)}{\text{vol}(\mathcal{E})} \leq e^{-1/(2n)}.$$

- After k iterations the volume is at most $e^{-k/(2n)}$ of the initial volume 
- To reach a fraction ϵ : $k \approx 2n \ln(1/\epsilon)$ iterations 
- Store $Q = \kappa LDL^\top$; the rank-one update keeps positive definiteness **by construction**



Section 4

From Magnitudes to Convexity

 Spectral Factorization Makes It Convex

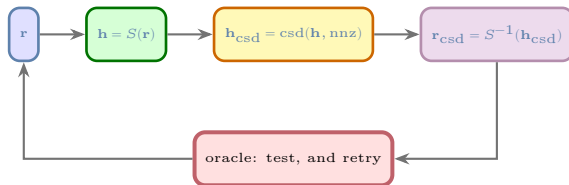
- The squared magnitude is a cosine series in the **autocorrelation**:

$$R(\omega) = r_0 + 2 \sum_{k=1}^{n-1} r_k \cos(k\omega) = \mathbf{a}(\omega)^\top \mathbf{r}.$$

- So the mask $L^2(\omega) \leq R(\omega) \leq U^2(\omega)$ is **convex in $\mathbf{r}$** ✓
- Spectral factorization recovers the minimum-phase h afterwards.



Quantization-Aware Design



- Constraints are tested against r_{csd} , a **realizable** design ✓
- An ineffective cut $\rightarrow$ perturb the quantization and **retry** ↺
- A sampled grid means a **discretization artifact** between samples ⚠

Section 5

Realization

 Spectral Factorization: FFT vs Roots

	Kolmogorov (FFT)	Aberth (roots)
needs FFT	yes	no
memory	$O(\text{over} \cdot N)$	$O(N)$
tuning	none	tolerance
use for	production	exploration

- Reconstruction: multiply well-separated roots first (**Leja**) → machine precision 🎯
- Scale by $s = \max(1, \max_k |a_k|)$ so the tolerance becomes **relative** 🔧

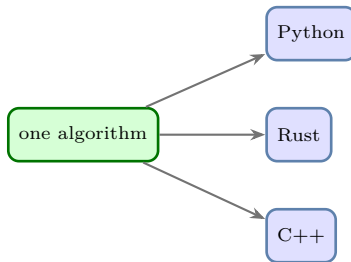


Direct vs Transposed Form

- **Direct form:** forms each product $h[k] x[n - k]$; exposes per-tap products.
- **Transposed form:** pipelines the accumulator with reversed coefficients; one output, one-cycle latency 
- Both use the **same CSD strings** $\rightarrow$ the same per-tap hardware 
- Decide by register count, adder depth, and wiring — not by quantization.

Section 6

Practice

 → → Three Implementations

- A shared ellipsoid engine, spectral factorization, and CSD quantizer 
- The oracle checks passband, stopband, and non-negativity **round-robin** and returns the first violation.



Cross-Language Results

Implementation	Mean time	Relative	Iterations
C++ (FFTW3)	303 ms	1.00×	1850
Rust (realfft)	286 ms	0.95×	2530
Python (NumPy)	4046 ms	13.4×	1693

- Compiled implementations within **5%**; interpreted **~13×** slower 🐢
- Different trajectories, but the **same energy** and the same specification ✅



Profiling Lessons

- Profile first: the **constraint scan** was 83%, the FFT only 6% 🔍
- One matvec + masks replaced ~500k tiny dots: **94×** on the primitive, 1.6× end-to-end 🚀
- The same change **regressed** compiled C++ by 1.9× (early exit beats a full matvec) ⚠️
- Rust: a scalar `.sum()` chain → a **4-accumulator** kernel ⚡
- **Port the measurement, not the conclusion.** 📌



Multiplierless Result

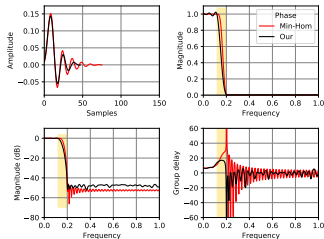


Figure 2: Magnitude response

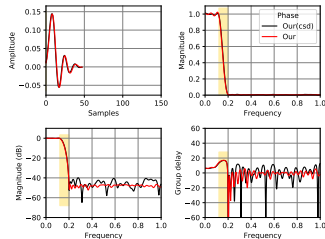


Figure 3: CSD-quantized response

- The quantizer is consulted **during** optimization, so the design is feasible by construction 

Section 7

Closing



Key Takeaways

The method 🧠

- the **oracle** is the star; the ellipsoid is bookkeeping
- **parallel cuts** pay for two-sided bounds

The practice 🛠️

- quantize **inside** the oracle — round-after fails
- profile, pre-allocate, and re-measure



References

- the paper: “Multiplierless FIR Filter Design with Parallel-Cut Ellipsoid Method” 
- Wu et al. (1999), Goodman (1997) — spectral decomposition and factorization 
- Bland, Goldfarb & Todd (1981) — the ellipsoid method 
- George (1960) — canonical signed digit 



Q&A

Multiplierless FIR Filter Design with Parallel-Cut Ellipsoid Method

Questions? Discussion? 



Thank You

Shift. Add. Cut. Repeat.   ▼

Slides built with Beamer · TikZ  · LuaLaTeX  · Nord 